Linux for DevOps

Lesson 2: File System & Permissions

File System & Permissions - Study Notes

Key Concepts

1. **Directory Hierarchy** – The Linux file system is organized in a tree rooted at `/`. Key top level directories such as `/etc`, `/var`, `/usr`, and `/home` have distinct purposes.

2. **File Ownership** – Every file/directory has an owner user (`uid`) and an owning group (`gid`). These determine who can read, write, or execute the file.

3. **Permission Bits** – Permissions are split into three sets (user, group, others) and three modes (read, write, execute). They can be displayed numerically (e.g., `755`) or symbolically (e.g., `rwxr-xr-x`).

4. **Changing Ownership** – The `chown` and `chgrp` commands alter the owner or group of a file or directory.

5. **Modifying Permissions** – The `chmod` command changes the permission bits, either via symbolic notation (`u+x`) or numeric notation (`chmod 644 file`).

Summary

Linux's file system is a hierarchical tree where each node is a file or directory. The top level directories have specific roles: `/etc` stores system configuration files, `/var` holds variable data such as logs and mail, `/usr` contains read only user data, and `/home` is where user home directories live. Understanding this structure helps you locate where to place files and where to find configuration settings.

Every file is owned by a user and a group, and the operating system uses this ownership to enforce access control. Permissions are expressed as read (`r`), write (`w`), and execute (`x`) bits for the owner, the group, and others. The `chmod` command changes these bits, while `chown` and `chgrp` change the ownership. Mastery of these tools lets you secure files, grant collaboration rights, and troubleshoot permission errors.

Important Points

• **Directory Roles**

- `/etc` – static system configuration files (e.g., `/etc/passwd`, `/etc/ssh/sshd_config`).
- `/var` – variable data (logs in `/var/log`, mail in `/var/mail`).
- `/usr` – secondary hierarchy for read only user data and executables.
- `/home` – per user home directories (`/home/alice`).

• **Permission Representation**

- Numeric: `r=4`, `w=2`, `x=1`. Sum per set gives a three digit octal number.
- Symbolic: `u` (user), `g` (group), `o` (others), `a` (all). Operators: `+`, `-`, `=`.

• **Common `chmod` Usage**

- `chmod 755 script.sh` !' owner full access, group/others read+execute.
- `chmod u+x file` !' add execute for the owner.

- `chmod go-w file` !’ remove write from group and others.

• **Changing Ownership**

- `chown alice file` !’ set owner to `alice`.
- `chown alice:staff file` !’ set owner to `alice` and group to `staff`.
- `chgrp staff file` !’ change group only.

• **Recursive Operations**

- `chmod -R 755 /path` – change permissions of all files/directories under `/path`.
- `chown -R alice:staff /path` – change ownership recursively.

• **Special Permissions**

- Setuid (`4` in the user execute bit) – program runs with the file owner’s privileges.
- Setgid (`2` in the group execute bit) – new files inherit the directory’s group.
- Sticky bit (`1` in the others execute bit) – only owner, root, or directory owner can delete/rename files in a directory (e.g., `/tmp`).

• **Common Pitfalls**

- Giving `777` to a directory grants everyone full control; use sparingly.
- Changing ownership of system files can break services; always double check.
- Remember that `chmod` affects only the target; `chown` does not change permissions.

Examples

1. Restricting Access to a Configuration File

```bash
# File: /etc/ssh/sshd_config

# Current permissions: -rw-r--r--

# Owner: root, Group: root

# Restrict to root only

sudo chmod 600 /etc/ssh/sshd_config
```

• **Explanation**: `600` sets read/write for owner only; removes all permissions for group and others. This prevents non root users from reading or modifying SSH configuration.

2. Sharing a Project Directory with a Team

```bash
# Create a group for the team

sudo groupadd devteam
```

Add users to the group

sudo usermod -aG devteam alice
sudo usermod -aG devteam bob

Create project directory

sudo mkdir /var/www/project
sudo chown root:devteam /var/www/project

Set permissions: group read/write/execute, others none

sudo chmod 770 /var/www/project

Enable setgid so new files inherit group

sudo chmod g+s /var/www/project
```

- **Explanation**:
  - `770` gives full access to owner (`root`) and group (`devteam`).
  - `g+s` ensures that files created inside inherit the `devteam` group, simplifying collaboration.

---

## Quick Review

1. What is the purpose of the `/etc` directory?
2. How do you interpret the permission string `-rwxr-x--x`?
3. Write the `chmod` command to give the owner read/write, the group read, and others no permissions.
4. How would you change the group of `/var/log/syslog` to `adm` while keeping the owner unchanged?
5. Explain the effect of setting the setgid bit on a directory.

---

## Further Reading

- **Advanced File Permissions** – ACLs (Access Control Lists) with `setfacl` and `getfacl`.
- **SELinux / AppArmor** – Mandatory Access Control systems for fine grained security.
- **Filesystem Hierarchy Standard (FHS)** – Detailed directory layout and conventions.
- **User and Group Management** – `useradd`, `usermod`, `/etc/passwd`, `/etc/group`.
- **File System Types** – ext4, XFS, Btrfs, and their specific features (e.g., snapshots).

---
```